

## Session 10

***1. Download a sample SRS (Software Requirements Specification) document for a food delivery app (like Zomato) and perform a peer review by highlighting at least 5 unclear or ambiguous requirements.***

### **Peer Review of SRS Document – Food Delivery App**

**Objective:**

To review an SRS document for a food delivery application and identify unclear or ambiguous requirements.

**Sample SRS Review**

| No | Requirement                                         | Problem Found                       | Suggested Clarification                                                                               |
|----|-----------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------------------------|
| .  |                                                     |                                     |                                                                                                       |
| 1  | "The app should load quickly."                      | "Quickly" is unclear.               | The home page should load within <b>3 seconds</b> under normal network conditions.                    |
| 2  | "Users can search for restaurants easily."          | "Easily" is subjective.             | Users should be able to search restaurants using restaurant name, cuisine, or location.               |
| 3  | "The system should send notifications immediately." | "Immediately" is ambiguous.         | Order status notifications should be sent within <b>10 seconds</b> of a status change.                |
| 4  | "The delivery should be fast."                      | No specific delivery time is given. | The application should display an estimated delivery time, for example <b>30–45 minutes</b> .         |
| 5  | "Users can make secure payments."                   | "Secure" is not defined.            | Payments should use the application's approved payment gateway and follow required security controls. |

### **Conclusion**

The peer review identified **5 ambiguous requirements**. Each requirement should contain measurable and specific information so that developers and testers can understand exactly what needs to be implemented and tested.

---

***2. Conduct an informal review of a friend's written test case set for a simple login feature (username and password), and write down 3 suggestions or improvements you would make.***

## **Informal Review of Login Test Cases**

**Feature:** Login with Username and Password

**Objective:**

To review a friend's test cases and suggest improvements.

### **Suggestions**

#### **Suggestion 1 – Add Negative Test Cases**

The test cases should include invalid username and password combinations.

**Example:**

Enter a valid username with an incorrect password.

**Expected Result:**

An appropriate error message should be displayed and the user should not be logged in.

---

#### **Suggestion 2 – Add Preconditions**

The test case should mention what needs to be true before testing.

**Example:**

- User should have a registered account.
- Application should be installed/open.
- Login page should be accessible.

---

#### **Suggestion 3 – Make Expected Results Specific**

Instead of writing:

"Login should work."

Write:

"The user should be successfully logged in and redirected to the home/dashboard page."

## Conclusion

The test cases can be improved by adding **negative scenarios, preconditions, and clear expected results.**

---

*3. Simulate a walkthrough session by explaining the steps of a test case for 'Add to Cart' functionality (like Flipkart) to a classmate and note any questions or feedback they give you.<br><br><em><strong>Hint:</strong></em> Focus on whether all steps are clear and if any preconditions or expected results are missing.*

# Walkthrough – Add to Cart Functionality

**Application:** Flipkart-style shopping app

**Feature:** Add to Cart

## Objective:

To explain the test case to a classmate and identify unclear steps or missing information.

## Test Case

**Test Case ID:** TC\_CART\_001

**Scenario:** Verify that a user can add a product to the shopping cart.

## Preconditions

1. User should have access to the shopping application.
2. Internet connection should be available.
3. At least one product should be available.
4. Product should be available for purchase.

## Steps

1. Open the shopping application.
2. Search for a product.
3. Select the required product from the search results.
4. Open the product details page.
5. Click the **Add to Cart** button.
6. Open the shopping cart.
7. Check the products listed in the cart.

## Expected Result

The selected product should be added successfully to the cart, and the cart should display the correct product name, quantity, and price.

## Simulated Classmate Feedback

During the walkthrough, the classmate gave the following feedback:

1. **Question:** What should happen if the product is out of stock?
  - **Improvement:** Add a negative test case for an out-of-stock product.
2. **Question:** Should the cart quantity increase when the same product is added again?
  - **Improvement:** Add a separate test case for adding the same product twice.
3. **Question:** What happens if the user clicks "Add to Cart" without selecting a required product option, such as size?
  - **Improvement:** Add a test case to verify validation for required product options.

## Conclusion

The walkthrough showed that the basic test case was understandable, but additional **preconditions, negative scenarios, and validation cases** would make it more complete.

---

*4. Identify 4 types of errors or issues that static testing (reviewing documents or code without execution) can catch in a test case document for an online movie booking system (like BookMyShow).*

# Errors/Issues Found Through Static Testing

**Application:** Online Movie Booking System like BookMyShow

Static testing means reviewing the **test case document, requirements, or code without executing the application.**

## Four Common Issues

| No. | Error/Issue                    | Example                                                                                                               |
|-----|--------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 1   | <b>Ambiguous requirement</b>   | "Movie should load quickly." There is no defined time.                                                                |
| 2   | <b>Missing test steps</b>      | Test case says "Book a ticket" but doesn't explain how to select movie, theatre, date, and seats.                     |
| 3   | <b>Missing expected result</b> | A test case describes the booking steps but does not state what should happen after successful booking.               |
| 4   | <b>Incomplete test data</b>    | The test case doesn't specify the movie, theatre, date, number of seats, or payment information required for testing. |

## Example of a Static Review

### Original test case:

"Select a movie and book tickets. Check that booking is successful."

### Problems found:

- Which movie should be selected?
- Which theatre should be selected?
- How many seats should be booked?
- What payment method should be used?
- What exactly confirms successful booking?

### Improved version:

"Select an available movie, choose a theatre and showtime, select 2 available seats, proceed to payment, complete the payment, and verify that a booking confirmation with the correct movie, theatre, showtime, and seat details is displayed."

## Final Summary

| Question | What You Completed                                      |
|----------|---------------------------------------------------------|
| 1        | Peer review of SRS with <b>5 ambiguous requirements</b> |
| 2        | Informal review with <b>3 improvements</b>              |

- 3 Walkthrough of **Add to Cart** test case + classmate feedback
- 4 **4 static testing issues** in a movie booking test document